

PROYECTO ARGUS

Sistema de Detección de Fatiga y Prevención de Accidentes en Transporte de Carga

Documento de Definición Técnica y Arquitectura de Solución

Preparado para: Comité de Titulación

Cumplimiento normativo: NOM-087

1. Resumen ejecutivo

Argus es un sistema ciberfísico distribuido diseñado para prevenir accidentes viales causados por fatiga y microsueños en operadores de transporte de carga. La solución combina procesamiento de visión artificial en el borde (edge computing) sobre Raspberry Pi 5, sensado físico de presión en el volante mediante ESP32, y una plataforma en la nube 100% serverless sobre AWS, orquestada con AWS CDK como Infraestructura como Código (IaC).

Este documento consolida en una sola fuente de verdad la definición técnica por módulo exigida por el comité (Arquitectura de Sistemas, Sistemas Inteligentes y Sistemas Distribuidos), la arquitectura de infraestructura AWS, y la especificación de requerimientos MVP, de forma que sirva como referencia única para el diseño detallado, la implementación y la evaluación del proyecto.

2. Diagrama de arquitectura semántica

El siguiente diagrama representa las tres capas del sistema: la zona Edge dentro del camión (captura, sensado y decisión local), la nube AWS Serverless (ingesta, lógica de negocio y persistencia), y el frontend React que actúa como Torre de Control para administradores y choferes.

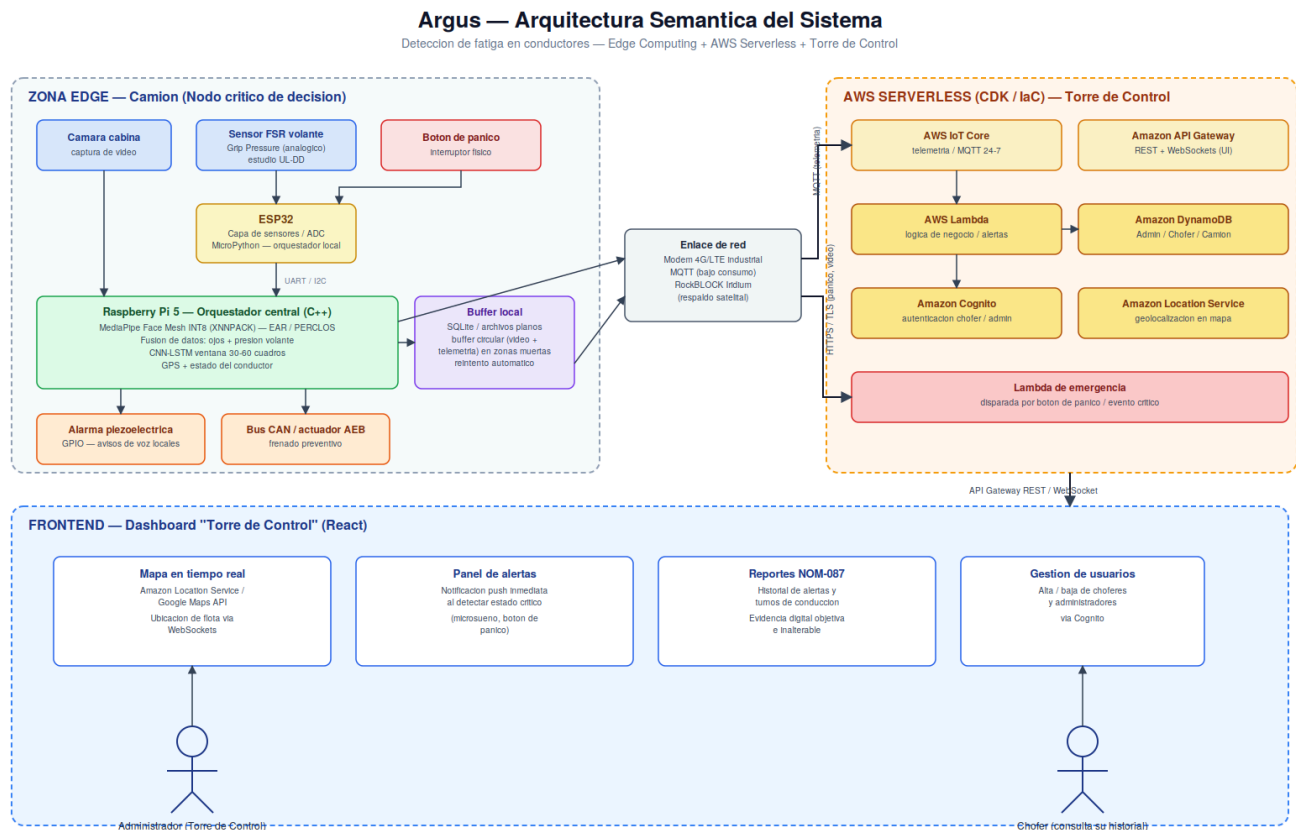


Figura 1. Arquitectura semántica de Argus — Edge, AWS Serverless y Torre de Control.

3. Arquitectura por capas

3.1 Capa Edge — Hardware en el camión

El procesamiento crítico ocurre localmente para garantizar baja latencia y continuidad de operación en zonas sin cobertura celular.

- **ESP32 (capa de sensores):** Lectura analógica (ADC) de los sensores de presión resistiva (FSR) en el volante; bajo consumo y alta precisión.
- **Raspberry Pi 5 (orquestador central):** Ejecuta el modelo MediaPipe Face Mesh cuantizado a INT8 con el delegado XNNPACK, en C++, para minimizar la latencia de inferencia sobre los 4 núcleos disponibles.
- **Fusión de datos:** La Raspberry Pi combina, mediante UART/I2C, la señal de presión del volante con las métricas oculares (EAR/PERCLOS) para una predicción de somnolencia más robusta que un solo canal de evidencia.
- **Comunicación saliente:** Envío de ubicación GPS y estado del conductor a AWS IoT Core vía MQTT, optimizando el consumo de datos en red celular 4G/LTE; RockBLOCK (Iridium) como respaldo satelital en rutas federales sin señal.

3.2 Software de borde y estructuras de datos

- **Lenguajes:** C++ para captura de video e inferencia (latencia mínima); Python/MicroPython para el orquestador ESP32 y manejo concurrente de sensores y MQTT.
- **Persistencia temporal:** Buffer circular en memoria volátil para video y telemetría durante "zonas muertas" de carretera, con sincronización automática al recuperar señal.
- **Memoria local persistente:** SQLite o archivos planos como respaldo local, con lógica de reintento automático hacia la nube.
- **Botón de pánico:** Un interruptor físico de alta prioridad conectado al ESP32 dispara directamente una Lambda de emergencia, sin pasar por la lógica normal de inferencia.

3.3 Backend — AWS Serverless (definido con AWS CDK)

Toda la infraestructura se define como Infraestructura como Código (IaC) con AWS CDK, permitiendo desplegar y replicar el entorno de forma reproducible.

- **AWS IoT Core:** Conexión persistente y ligera para telemetría 24/7 desde el camión (GPS, estado del conductor, heartbeats).
- **Amazon API Gateway:** Expone los endpoints REST que consume la UI de React para consultar datos históricos y gestionar usuarios; soporta WebSockets para actualizar la ubicación del camión en el mapa en tiempo real.
- **AWS Lambda:** Procesos asíncronos: procesar una alerta de emergencia, disparar notificaciones push a administradores, generar reportes de fatiga/rendimiento.
- **Amazon DynamoDB:** Base de datos NoSQL para las entidades Admin, Chofer y Camión (ver modelo de datos en la sección 3.4).
- **Amazon Cognito:** Autenticación de choferes y administradores; protege el acceso a la UI y a las APIs.
- **Comunicaciones seguras:** Comunicación entre el camión y la nube (HTTPS/TLS de la Raspberry Pi a AWS) y entre el camión y sus periféricos (UART/I2C ESP32↔Pi; CAN bus hacia el sistema AEB del camión).

3.4 Modelo de datos — Amazon DynamoDB

Se modelan tres entidades principales como base para definir posteriormente el esquema de partición (PK) y clave de ordenamiento (SK):

- **Entidad Admin:** Personal en la Torre de Control con permisos globales de gestión de flota y usuarios.
- **Entidad Chofer:** Datos biométricos de referencia, historial de alertas y turnos de conducción, necesarios para verificar el cumplimiento de la NOM-087.
- **Entidad Camión:** Identificador de placa, ubicación en tiempo real y estado crítico (normal / advertencia / microsueño).

Pendiente de definición como siguiente paso: esquema detallado de claves de partición (PK) y ordenamiento (SK) por entidad, y patrones de acceso (access patterns) que optimicen las consultas del dashboard.

3.5 Frontend — Torre de Control (React)

Interfaz tipo dashboard de operaciones para los administradores de flota:

- **Mapa en tiempo real:** Visualización de la flota sobre Amazon Location Service o Google Maps API, actualizada vía WebSockets/MQTT.
- **Panel de alertas:** Notificaciones visuales inmediatas cuando un camión entra en estado crítico (microsueño detectado).
- **Reportes:** Historial de alertas y turnos como evidencia digital objetiva e inalterable, alineado a NOM-087.
- **Seguridad:** Autenticación de choferes y administradores mediante Amazon Cognito.

4. Especificación de requerimientos (MVP)

Tabla consolidada requerimiento por requerimiento, fusionando la especificación funcional original con la arquitectura AWS/Edge definida.

ID	Requerimiento	Solución técnica	Tecnología / Servicio	Módulo
R1	Detección de somnolencia ocular	Extracción de 68 landmarks faciales; cálculo de EAR en tiempo real; clasificación PERCLOS con umbral ≥ 1.5 s como microsueño; ventana temporal de 30-60 cuadros analizada con regresión logística o CNN-LSTM.	MediaPipe Face Mesh INT8 + XNNPACK sobre Raspberry Pi 5 (4 núcleos)	Mód. 2
R2	Detección conductual complementaria	Sensores FSR en el volante detectan relajación muscular como indicador temprano de fatiga; se fusiona con la señal ocular en la Raspberry Pi.	ESP32 + FSR (estudio UL-DD), UART/I2C	Mód. 1 / 2
R3	Alertas en cabina	Activación de alarma sonora y avisos de voz procesados localmente, sin dependencia de red, ante evento de fatiga confirmado.	GPIO (Raspberry Pi / ESP32), audio local	Mód. 1
R4	Frenado preventivo	Interfaz con el bus CAN del camión o un actuador electrónico para activar el sistema de frenado autónomo de emergencia.	Bus CAN, AEB	Mód. 1
R5	Botón de pánico	Interruptor físico de alta prioridad en el ESP32 que dispara de inmediato una función Lambda de emergencia en la nube.	ESP32 → MQTT → AWS Lambda	Mód. 1 / 3
R6	Transmisión celular de telemetría	Envío continuo y de bajo consumo de datos de ubicación y estado del conductor hacia AWS IoT Core.	Módem 4G/LTE industrial, protocolo MQTT	Mód. 3
R7	Continuidad sin cobertura	Buffer circular local de video/telemetría en zonas muertas; sincronización automática al recuperar señal; enlace satelital como respaldo en rutas federales.	SQLite / archivos planos + RockBLOCK (Iridium)	Mód. 1 / 3
R8	Plataforma cloud	Backend 100% serverless: ingesta vía IoT Core, lógica de negocio en Lambda, persistencia en DynamoDB, exposición de API vía API Gateway; toda la infraestructura definida con AWS CDK (IaC).	AWS Lambda + DynamoDB + API Gateway + IoT Core (CDK)	Mód. 3
R9	Autenticación y control de acceso	Gestión de identidades para choferes y administradores, protegiendo el acceso a la UI y a las APIs.	Amazon Cognito	Mód. 3
R10	Torre de Control (dashboard)	Mapa de flota en tiempo real, panel de alertas críticas y reportes históricos de fatiga/turnos como evidencia digital para NOM-087.	React + Amazon Location Service / Google Maps API + WebSockets	Mód. 3

ID	Requerimiento	Solución técnica	Tecnología / Servicio	Módulo
R11	Sincronización en tiempo real de UI	Actualización instantánea de la ubicación del camión y del estado del conductor en el dashboard de administradores.	WebSockets + MQTT	Mód. 3
R12	Segmentación de carriles (mejora opcional)	Detección de bordes y transformada de Hough para mantener el camión centrado durante el frenado autónomo.	Canny Edge Detection + Transformada de Hough	Mód. 2 (opcional)
R13	Monitoreo biométrico multimodal (mejora opcional)	Fusión intermedia de señales de ritmo cardíaco y respiración junto con visión y presión de volante, para elevar la precisión de detección de fatiga hasta ~84%.	Sensores biométricos adicionales + fusión de sensores	Mód. 2 (opcional)

5. Mapeo a criterios de evaluación del Comité

5.1 Módulo 1 — Arquitectura y Programación de Sistemas

- **1.1 / 1.2 Lenguajes y desempeño:** C++ en el Edge para inferencia de baja latencia; Python/MicroPython en el orquestador ESP32.
- **Estructuras de datos:** Buffer circular local (memoria volátil) para tolerancia a desconexión de red.
- **1.5 Modelado del sistema:** Diseño modular Edge (nodo crítico de decisión) vs. Nube (torre de control administrativa), definido como IaC con AWS CDK.

5.2 Módulo 2 — Sistemas Inteligentes

- **2.1.2 / 2.1.3 Visión artificial y ML:** EAR con 68 landmarks faciales y clasificación PERCLOS (umbral ≥ 1.5 s).
- **2.2 Modelo matemático:** Regresión logística o CNN-LSTM sobre ventana temporal de 30-60 cuadros.
- **2.3 Justificación de algoritmos:** MediaPipe Face Mesh (INT8) elegido por su desempeño en hardware limitado con alta precisión y baja tasa de falsos positivos ante variaciones de iluminación.

5.3 Módulo 3 — Sistemas Distribuidos

- **3.2 Modelo cliente-servidor:** Camión como cliente inteligente que consume AWS IoT Core (telemetría) y API Gateway (gestión de usuarios).
- **3.1.6 Sincronización en tiempo real:** WebSockets y MQTT para actualizar la ubicación del camión en la UI de forma instantánea.
- **3.3 Comunicación entre dispositivos:** UART/I2C entre ESP32 y Raspberry Pi; HTTPS/TLS entre Raspberry Pi y la nube.

6. Mejoras técnicas (módulos opcionales)

- **Sensor de presión del volante:** Sensores FSR en el volante para detectar relajación muscular como indicador temprano de fatiga conductual (estudio UL-DD).
- **Segmentación de carriles:** Canny Edge Detection y Transformada de Hough para mantener el camión centrado durante el frenado autónomo.
- **Monitoreo biométrico multimodal:** Fusión intermedia de ritmo cardíaco y respiración junto a la señal visual, alcanzando hasta ~84% de precisión en detección de fatiga.
- **Comunicación satelital:** Módulo RockBLOCK (Iridium) como enlace de respaldo para reportar ubicación exacta en rutas federales sin cobertura.

7. Próximos pasos

- **1.** Definir el esquema detallado de DynamoDB (PK y SK) para las entidades Admin, Chofer y Camión, junto con los access patterns del dashboard.
- **2.** Detallar los diagramas de secuencia por caso de uso crítico (microsueño detectado, botón de pánico, pérdida y recuperación de conectividad).
- **3.** Especificar los contratos de los endpoints de API Gateway y los topics de MQTT en IoT Core.
- **4.** Definir el stack de AWS CDK (stacks, constructs y variables de entorno) por ambiente (dev/staging/prod).

Documento generado a partir de la fusión de la definición técnica por módulos y la arquitectura de infraestructura AWS/CDK, como referencia única para el diseño detallado del proyecto Argus.